OS Orange Problem-2

PES-VCS: Local Version Control System Report

Name: Sujay Hegde **SRN:** PES1UG24CS478

Sec: 4-H

Repository: https://github.com/sujayhegde23/PES1UG24CS478-pes-vcs

Phase 1: Object Store (`object.c`)

Description

Implemented a content-addressable storage system where every file is stored by its SHA-256 hash. Same content = same hash = stored only once (deduplication).

Functions Implemented

`object_write` — Store data in the object store

- Builds header string (`"type size\0"`)
- Concatenates header + data into full object
- Computes SHA-256 hash of the full object
- Checks for deduplication (skips if object already exists)
- Creates shard directory (`.pes/objects/XX/`)
- Writes to temporary file with `fsync()` for durability
- Atomic rename from temp to final path

`object_read` — Read and verify data from the object store

- Builds file path from hash

- Reads entire file into memory
- Verifies integrity by recomputing SHA-256 and comparing
- Parses header to extract type (blob/tree/commit)
- Returns data portion after the null terminator

Test Results

Screenshot 1A: Output of `./test_objects` showing all tests passing

```
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$ ./test_objec
ts
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9
db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227e
d0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: Sharded directory structure (`find .pes/objects -type f`)

```
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$ find .pes/ob
jects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9
aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e
566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11e
c4f476
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$
```

Phase 2: Tree Objects (`tree.c`)

Description

Implemented tree objects that represent directory snapshots. Each tree entry points to a blob (file) or another tree (subdirectory), enabling hierarchical storage.

Functions Implemented

`tree_from_index` — Build tree hierarchy from staged files

- Sorts index entries by path for correct grouping
- Recursively builds subtrees for nested directories
- Files at current level are added as blob entries
- Subdirectories are recursively processed into tree objects
- Serializes and writes tree objects to the object store

Helper Functions

- `compare_entries_by_path` — Sort comparator for index entries
- `build_tree_recursive` — Recursive tree construction from flat path list

Test Results

Screenshot 2A: Output of `./test_tree` showing all tests passing

```
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Raw binary format of a tree object (xxd)

```
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$ xxd .pes/obj
ects/XX/YYY... | head -20
00000000: 7472 6565 2034 3930 3130 3036 3434 2066  tree 49.100644 f
00000010: 696c 6531 2e74 7874 002c f8d8 3d9e e295  ile1.txt..,..=...
00000020: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337  C..J.rt!........7
00000030: 6390 25de 576d b9eb b4                   c.%.Wm...
```

Phase 3: Staging Area (`index.c`)

Description

Implemented the staging area (index) that tracks which files are ready to be committed. The index is stored as a text file at `.pes/index`.

Functions Implemented

`index_load` — Read the index file into memory

- Opens `.pes/index` for reading
- Parses each line: `<mode> <hash> <mtime> <size> <path>`
- Handles missing index file gracefully (empty index for first use)

`index_save` — Write the index atomically

- Sorts entries by path using `qsort()`
- Writes to temporary file `.pes/index.tmp`
- Flushes with `fflush()` and `fsync()` for durability
- Atomic rename from temp to final index file

`index_add` — Stage a file for commit

- Reads file contents from working directory
- Creates blob object via `object_write()`
- Gets file metadata via `stat()` (mode, mtime, size)
- Updates existing entry or adds new entry to index
- Saves updated index to disk

Test Results

Screenshot 3A: Output of `pes init` + `pes add` + `pes status`

```
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index    # Human-readable text format
gcc -Wall -Wextra -O2 -c index.c -o index.o
index.c: In function 'index_add':
index.c:254:9: warning: implicit declaration of function 'object_write'
[-Wimplicit-function-declaration]
  254 |     if (object_write(OBJ_BLOB, data, size, &id) != 0) {
      |         ^~~~~~~~~~~~
index.c:250:5: warning: ignoring return value of 'fread' declared with a
ttribute 'warn_unused_result' [-Wunused-result]
  250 |     fread(data, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~~
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
```

```
Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.h
  untracked:  pes.c
  untracked:  README.md
  untracked:  test_tree
  untracked:  object.c
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  tree.h
  untracked:  file2.txt
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  Makefile
  untracked:  file1.txt
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  index.h
  untracked:  pes.h
  untracked:  tree.c
```

Screenshot 3B: Contents of `.pes/index` showing text-format entries

```
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$  cat .pes/in
dex

100644 2cf8d83d9ee29543b34a87727421fdec7be3f3a183d337639025de576db9ebb4
1776253890 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
1776253890 6 file2.txt
```

Phase 4: Commits (`commit.c`)

Description

Implemented the commit system that ties everything together — commits point to a tree (project snapshot) and contain metadata (author, timestamp, message, parent commit).

Functions Implemented

`commit_create` — Create a new commit

- Builds tree from current index via `tree_from_index()`
- Reads current HEAD as parent (handles first commit with no parent)
- Sets author from `PES_AUTHOR` environment variable
- Sets timestamp to current Unix time
- Serializes commit to text format
- Writes commit object to the object store
- Updates HEAD to point to the new commit

Test Results

Screenshot 4A: `pes log` showing commit history with multiple commits

```
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$  ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log

Committed: d0220fb07627... First commit
Committed: 2c6379c4f0bc... Add file3
Committed: 94562ccdb9e0... Update file1
commit 94562ccdb9e0f6376cd4cb8dd91569fcdce28002b620b5802c57217c6ec23e50
Author: Sujay Hegde<PES1UG24CS478>
Date:   1776253996
    Update file1

commit 2c6379c4f0bc137e0a92f3039970c600f460ebaa102f63ae9a17ecf5ba8880d2
Author: Sujay Hegde<PES1UG24CS478>
Date:   1776253996

    Add file3

commit d0220fb07627619d8d3f61a57ca32a7b05084f8761d05a9ffc16393ed61ef9ac
Author: Sujay Hegde<PES1UG24CS478>
Date:   1776253996

    First commit
```

Screenshot 4B: Object store growth (`find .pes -type f | sort`)

```
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$ find .pes -t
ype f | sort
.pes/HEAD
.pes/index
.pes/objects/24/12cce4b97c09ebb1b604884ca66f141c2364a05858fef8ac4859da40
474bec
.pes/objects/2c/6379c4f0bc137e0a92f3039970c600f460ebaa102f63ae9a17ecf5ba
8880d2
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7be3f3a183d337639025de576d
b9ebb4
.pes/objects/64/5b507be548973ff35eab64c5e6efb7dd5e7d419484fafe519f60dd4b
88d889
.pes/objects/94/562ccdb9e0f6376cd4cb8dd91569fcdce28002b620b5802c57217c6e
c23e50
.pes/objects/d0/220fb07627619d8d3f61a57ca32a7b05084f8761d05a9ffc16393ed6
1ef9ac
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606
199f7f
.pes/objects/e9/37ea375e05180cda902480ca6a1e073c6182426466e05bf019b997ce
1c2324
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3ed6d9c4de7
```

Screenshot 4C: Reference chain (`cat .pes/refs/heads/main` and `cat .pes/HEAD`

```
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$ cat .pes/ref
s/heads/main
ref: refs/heads/main
94562ccdb9e0f6376cd4cb8dd91569fcdce28002b620b5802c57217c6ec23e50
sujay-hegde@sujay-hegde-VirtualBox:~/PES1UG24CS478-pes-vcs$ cat .pes/HEA
D
d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 5: Written Questions

Q5.1: How would you implement `pes checkout <branch>`?

To checkout a branch:

1. **Read the target branch** file from `.pes/refs/heads/<branch>` to get its commit hash.
2. **Update `.pes/HEAD`** to `ref: refs/heads/<branch>`.
3. **Read the commit object** → get the tree hash → recursively read the tree.
4. **Update the working directory** files to match the tree snapshot.
5. **Update the index** to match the new tree.

Complexity: The hard part is handling conflicts — if the user has uncommitted changes to files that differ between branches, checkout must refuse. You'd also need to handle file creation/deletion when switching between trees with different file sets.

Q5.2: How to detect uncommitted changes using only the index and object store?

1. For each entry in the index, `stat()` the working directory file.
2. Compare `mtime` and `size` from `stat` with the index entry's stored values.
3. If metadata differs, re-hash the file contents and compare with the stored blob hash.
4. If the hash differs, the file has uncommitted changes → refuse checkout.
5. Also check for deleted files (`stat` fails) and new untracked files.

Q5.3: What happens with commits in detached HEAD state?

In detached HEAD, `HEAD` contains a commit hash directly (not `ref: refs/heads/...`). New commits still work — each new commit's hash replaces the old hash in HEAD.

However, if you then checkout a branch, the detached commits become **"orphaned"** — no branch points to them. They're unreachable but still exist in the object store.

To recover: Note the commit hash before switching, then create a branch pointing to it:

```
git branch rescue-branch <hash>
```

Q6.1: Describe an algorithm to find and delete unreachable objects (Garbage Collection)

1. **Mark phase:** Start from all branch tips (files in `refs/heads/`). For each reachable commit, mark it and follow: commit → tree → recursively mark all subtrees and blobs. Use a **hash set** to track reachable hashes.

2. **Sweep phase:** Walk all objects in `.pes/objects/`. Delete any object whose hash is NOT in the reachable set.

3. **Estimation:** For 100K commits with 50 branches, you'd visit ~100K commits + their trees. If each commit has ~100 files, that's ~10M tree/blob references to check (but deduplication means far fewer unique objects).

Q6.2: Why is concurrent GC + commit dangerous?

Race condition: A commit operation creates objects in stages — first blobs, then trees, then the commit object, then updates the ref. If GC runs between "write tree" and "update ref," it could see the new tree as unreachable (no ref points to it yet) and delete it. When the commit operation tries to update the ref, it points to a deleted object.
Git's solution: Git uses a lock file (`.git/gc.lock`) and a grace period — GC only deletes objects older than **2 weeks** by default (`gc.pruneExpire`). This ensures in-progress operations complete before their objects become eligible for deletion